

CHAPTER I

INTRODUCTION TO DATA

SERIALIZATION

Assoc. Prof. Dr. Ngamnij Archint

CP363205 Data Serialization Languages and Applications





CONTENT OUTLINE

- What is Data Serialization vs Data Deserialization?
- Applications for Data Serialization
- Problems for Data exchange between computers
- Limitation of HTML
- Dynamic Web pages and Applications for Mobile Devices
- Text-based Data Serialization format (XML, JSON, YAML)
- Programming Languages supported for Texted-based Data Serialization

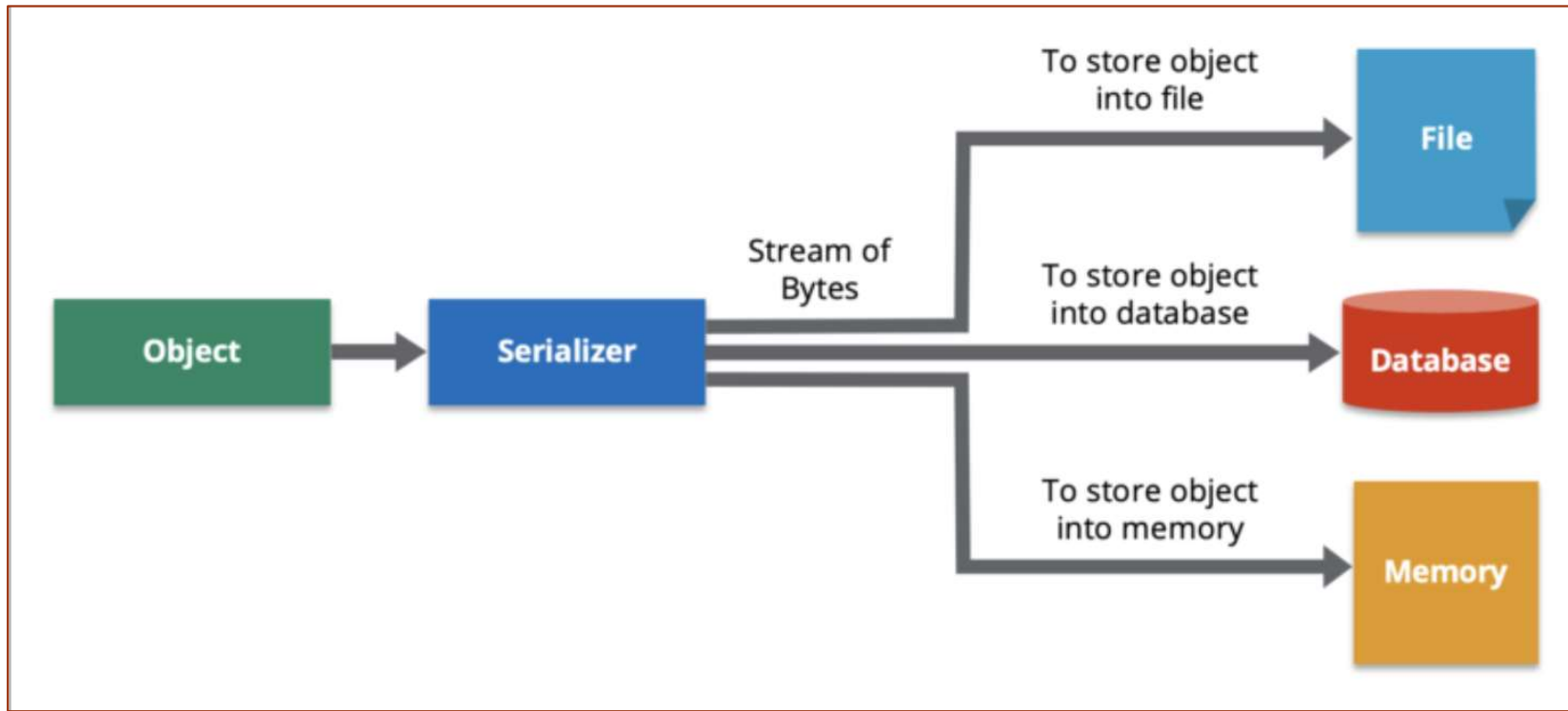


DATA SERIALIZATION คืออะไร

คือ กระบวนการในการแปลง **data objects** ที่อยู่ในโครงสร้างที่ซับซ้อน ที่ถูกส่งจากเครื่องคอมพิวเตอร์หรืออุปกรณ์ทางคอมพิวเตอร์ต้นทาง โดยแปลงให้อยู่ในรูปแบบของ**ข้อมูลมาตรฐานกลาง (data serialization formats)** ที่อยู่ในรูปแบบของ **text** หรือ **binary** เพื่อให้เหมาะสำหรับการจัดเก็บข้อมูลและการส่งข้อมูลไปยังเครื่องคอมพิวเตอร์หรืออุปกรณ์ทางคอมพิวเตอร์ปลายทางที่แตกต่างกันได้



DATA SERIALIZATION คืออะไร



<https://hazelcast.com/glossary/serialization/>

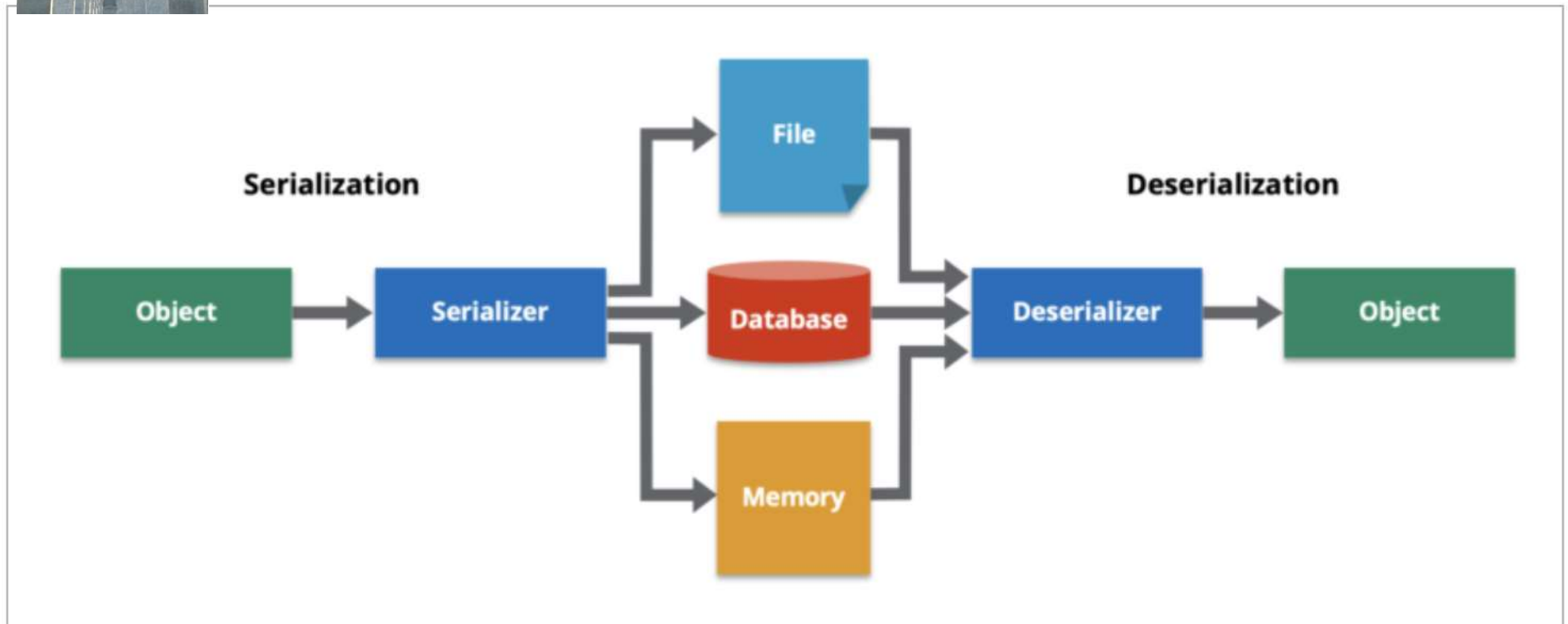


DATA DESERIALIZATION คืออะไร

คือ กระบวนการในการแปลงกลับจากข้อมูลมาตรฐานกลาง (data serialization formats) ที่ได้รับจากระบบคอมพิวเตอร์หรืออุปกรณ์ทางคอมพิวเตอร์ต้นทางให้อยู่ในรูปแบบของ Object ที่มีรูปแบบเหมือนกับ Object ที่ถูกสร้างอยู่บนเครื่องต้นทาง



DATA DESERIALIZATION คืออะไร



<https://hazelcast.com/glossary/serialization/>



APPLICATIONS ใหนบ้างที่มีการใช้ DATA SERIALIZATION

1. Storing data into Databases

- เมื่อ objects ในโปรแกรมภาษา **Java** ถูกแปลงให้เป็น **byte streams** และถูกเก็บใน **DBs (MySQL)** โดยผ่านทาง **Java JDBC**

2. Transfer data to the network

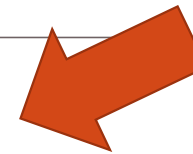


- เช่น web applications และ mobile apps ที่มีการส่งข้อมูลที่อยู่ในรูปแบบของ **objects** จาก **client** ไปยัง **server** หรือจาก **server** ไปยัง **client** เป็นต้น



APPLICATIONS โหนดบางที่มีการใช้ DATA SERIALIZATION

3. Sharing data in a Distributed Object Model



- เมื่อต้องการ **share object data** ระหว่างเครื่องคอมพิวเตอร์บนเครือข่าย ที่มี platforms ที่แตกต่างกัน โดยผ่านเทคโนโลยีเช่น **Web services** ที่ใช้ **SOAP, REST** เป็นตัวกลางในการรับส่งข้อมูล

4. Remote Method Invocation(RMI)

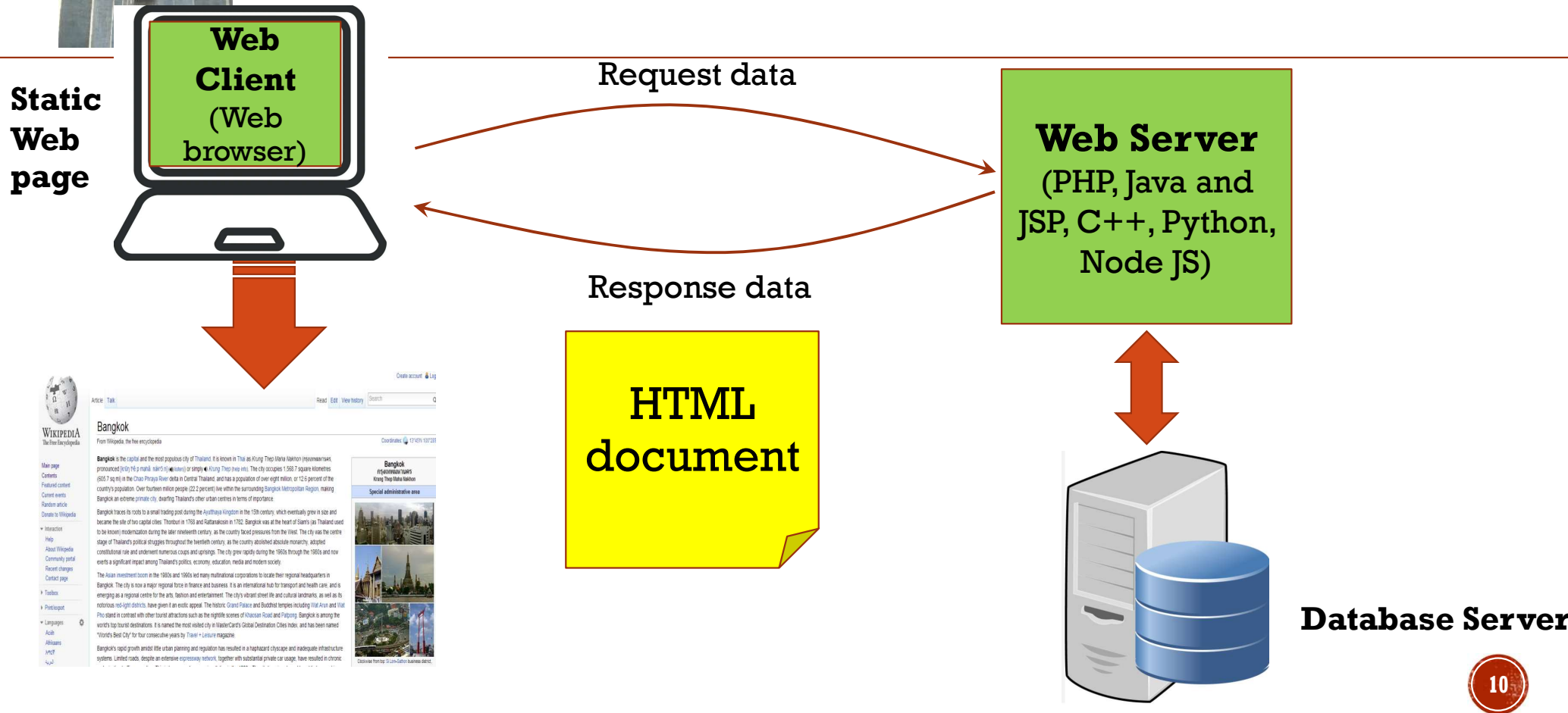
- โดยการส่ง **serialized objects** ในลักษณะของ parameters ไปยัง functions ที่ run อยู่บนเครื่องที่เป็น **remote machine**



ปัญหาการแลกเปลี่ยนข้อมูล ระหว่างระบบคอมพิวเตอร์

- เนื่องจากระบบคอมพิวเตอร์ต่าง ๆ จะมีความแตกต่างกันในด้านการออกแบบทาง ฮาร์ดแวร์ การใช้ ระบบปฏิบัติการ(OS) ที่แตกต่างกัน โครงสร้างการจัดเก็บข้อมูลที่แตกต่างกัน หรือแม้แต่การใช้ Applications ที่เขียนด้วยภาษาโปรแกรมที่แตกต่างกัน
 - ในการจะแลกเปลี่ยนข้อมูลกันระหว่างระบบคอมพิวเตอร์ที่แตกต่างกันหรือระหว่าง Applications ที่แตกต่างกันนี้จึงเป็นเรื่องที่ยาก
 - จึงมีความต้องการสร้างรูปแบบของข้อมูลที่เป็นมาตรฐานที่จะทำให้ระบบคอมพิวเตอร์ทุกระบบสามารถเข้าใจรูปแบบข้อมูลที่เป็นแบบเดียวกันได้

การแลกเปลี่ยนข้อมูลบนเทคโนโลยี WWW





การแลกเปลี่ยนข้อมูลบนเทคโนโลยี WWW

- เทคโนโลยี **WWW** จะสามารถรับส่งข้อมูลระหว่างเครื่องคอมพิวเตอร์ที่ทำหน้าที่เป็น **Web client** ที่เป็นเครื่องที่ให้ผู้ใช้งานทั่วไปสามารถใช้งานได้ กับอีกเครื่องที่เรียกว่า **Web server** จะมีเฉพาะผู้บริหารระบบงานคอมพิวเตอร์ที่จะใช้งานได้เพราะสามารถติดต่อกับฐานข้อมูลได้โดยตรง
- เครื่องทั้งสองฝั่งนี้อาจมี platform ที่แตกต่างกัน เช่น **Web client** อาจเป็นเครื่อง **PC** หรือ **notebook** และอาจมีการใช้ระบบปฏิบัติการที่แตกต่างกัน เป็น **MacOS, Window OS, Linux** เป็นต้น เครื่องที่ฝั่ง **server** ก็อาจเป็นเครื่องคนละแบบเป็นเครื่องขนาดใหญ่ที่มีระบบ OS ที่แตกต่างกันอีก เช่น **Linux, Unix** เป็นต้น



การแลกเปลี่ยนข้อมูลบนเทคโนโลยี WWW

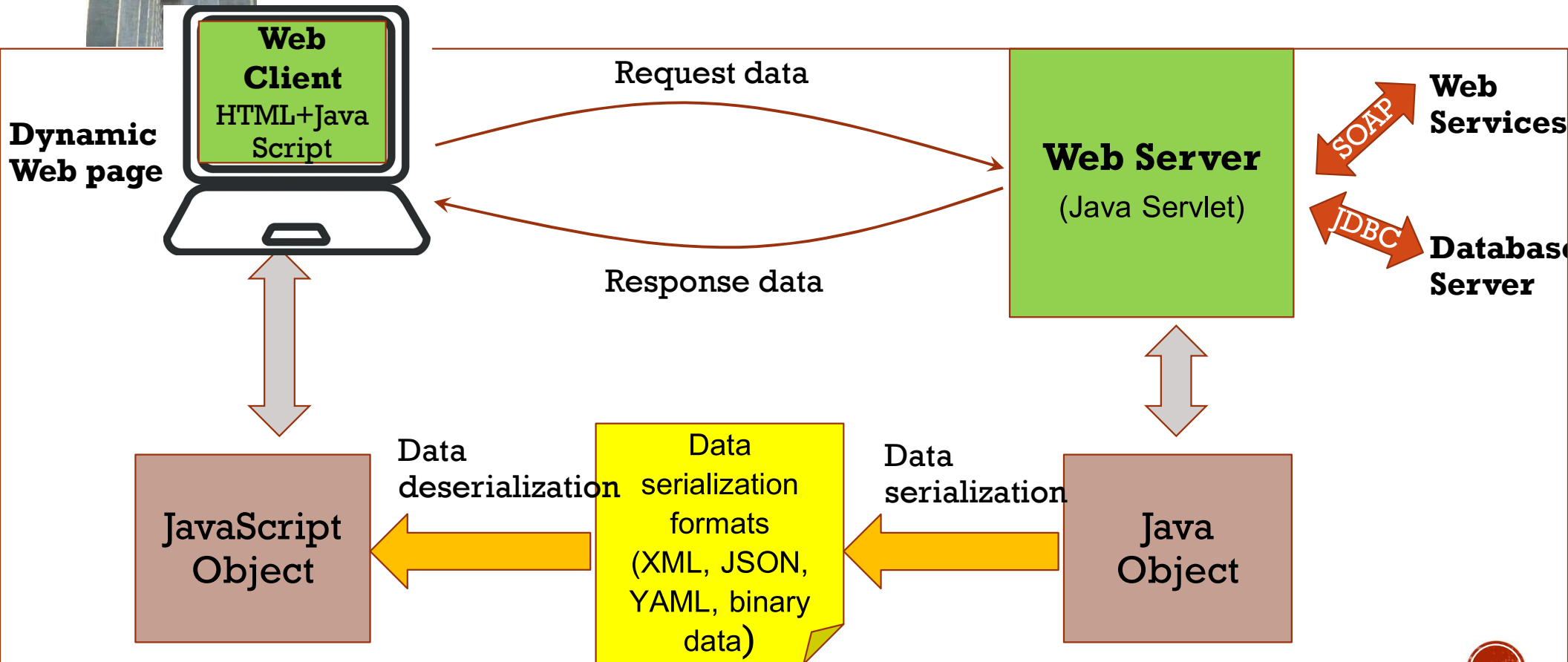
- การใช้เทคโนโลยีของ **WWW** ต้องเริ่มจากการที่ Web client ต้องมีการส่งคำร้องขอ หรือที่เรียกว่า **request** ไปยัง **web server** เช่น อาจมีการระบุ **url** บน Browser และ กด Enter url นั้น ก็จะถือว่าเป็น request ที่ต้องถูกส่งไปยัง server เพื่อบอก server ว่า client มีความต้องการร้องขอเอกสารที่เก็บอยู่ใน server และ **application** ทางฝั่ง **server** ก็จะรับ request นั้นเข้ามาเพื่อประมวลผลหรือดึงข้อมูลจาก **DB** และแปลงให้อยู่ในรูปแบบภาษา **HTML** และส่งไปยังเครื่องฝั่ง **client** ในลักษณะของ **response data** แต่ผลลัพธ์ที่ได้ที่ user เห็นบนหน้าเว็บก็จะมีลักษณะเป็นหน้าเว็บที่เป็น **static web page** นั้นหมายถึงผู้ใช้จะไม่สามารถแก้ไขข้อมูลในเว็บเพจนั้นได้ หรือนำข้อมูลที่แสดงนั้นไปใช้งานต่อในการประมวลผลในเรื่องอื่นๆจะทำได้ไม่ได้ เพราะ HTML นั้นจะเน้นที่การนำเสนอรูปแบบการแสดงผลเท่านั้น



ข้อจำกัดของการแลกเปลี่ยนข้อมูลด้วย HTML

- ข้อจำกัดของเอกสาร HTML
 - **HTML** ถูกออกแบบสำหรับการจัดรูปแบบเอกสารบนเว็บ โดยเน้นการนำเสนอเนื้อหาบนเว็บ (Information Presentation) แต่ไม่ได้เน้นการจัดเก็บข้อมูลเพื่อให้สามารถนำข้อมูลนั้นไปประมวลผลด้านอื่นต่อไปได้

การสร้าง DYNAMIC WEB PAGES





การสร้าง DYNAMIC WEB PAGES

- ถ้าต้องการสร้าง **dynamic web page** ซึ่งหมายถึง output data ที่เราได้รับจาก server จะมีลักษณะที่เป็น **dynamic** คือเป็นข้อมูลที่ client สามารถนำไปใช้ประมวลผลหรือแสดงผลในรูปแบบใดก็ได้ตามที่ client ต้องการโดยอิสระจาก server หรือเป็นเว็บที่เปิดโอกาสให้ผู้ใช้สามารถสร้าง content บนหน้าเว็บได้ด้วยตนเอง ไม่ใช่การอ่านได้อย่างเดียวเหมือน HTML
- จึงจำเป็นที่จะต้องสร้างภาษากลางที่เป็นมาตรฐานขึ้นมาใหม่แทน HTML ที่มีรูปแบบที่เรียกว่า **Data serialization formats**



การสร้าง DYNAMIC WEB PAGES

- **Data serialization formats** จะทำให้ข้อมูลที่ดึงได้จากฝั่ง server ที่อาจอยู่ในรูปแบบของ **object** เช่น ถ้าทาง server เขียนโปรแกรมด้วยภาษา Java ข้อมูลที่เก็บก็จะอยู่ในรูปแบบของ **Java object** เมื่อต้องการที่จะส่งข้อมูลนั้นไปยัง client ข้อมูล **java object** นั้น จะต้องถูกแปลงให้อยู่ใน **รูปแบบภาษากลาง** ซึ่งในปัจจุบันจะมีภาษาที่อยู่ในรูปแบบของ data serialization นี้หลายภาษา เช่น **XML, JSON, YAML** เป็นต้น ซึ่งเป็นภาษาที่มนุษย์สามารถอ่านเข้าใจได้

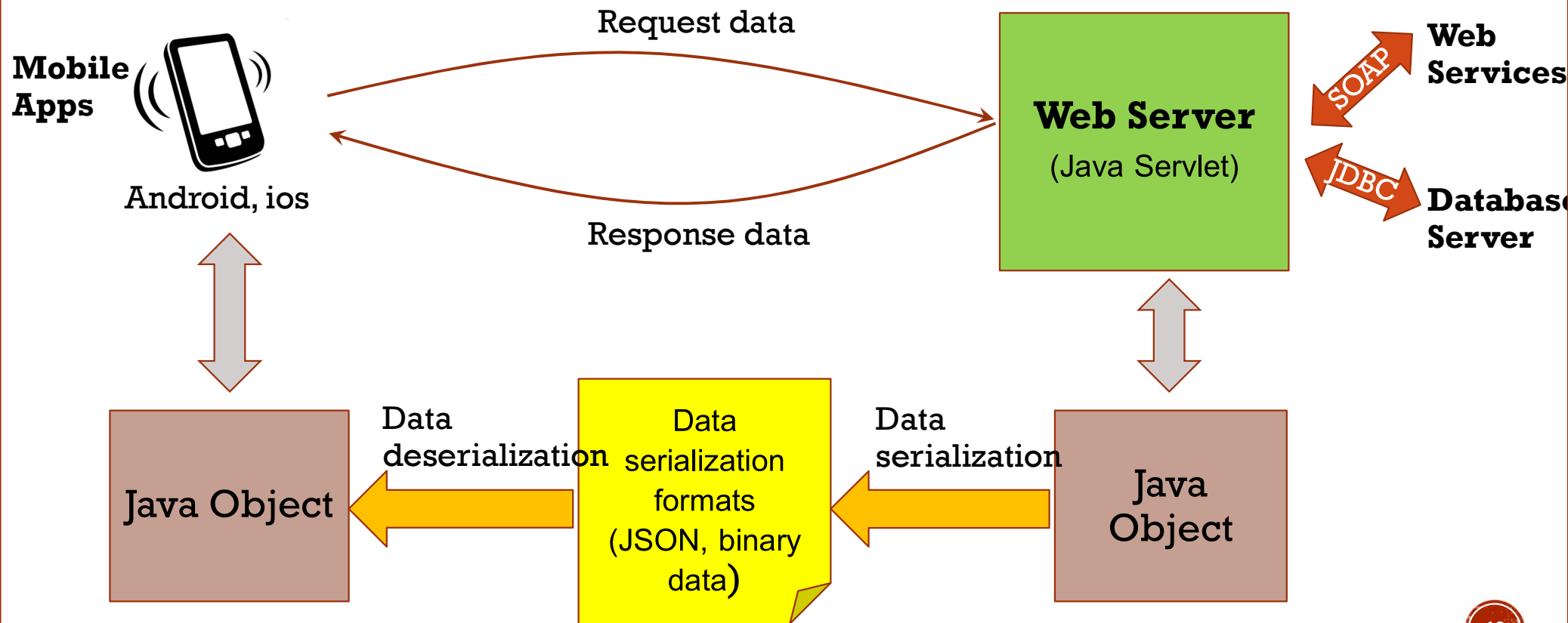


การสร้าง DYNAMIC WEB PAGES

- ดังนั้น กระบวนการที่เรียกว่า **Data Serialization** จะเป็นกระบวนการที่จะแปลงข้อมูลจากต้นทางซึ่งอาจอยู่ในรูปแบบของ **object data** ให้อยู่ในรูปแบบของ ภาษากลางที่มนุษย์อ่านได้ เช่น **XML, JSON, YAML** หรืออยู่ในรูปแบบภาษาที่มนุษย์อ่านไม่ได้ ที่เรียกว่า **binary data** ก็ได้ เช่น ภาษา **protobuf, apache thrift** เพื่อให้สามารถส่งไปในเครือข่ายและเมื่อข้อมูลนี้ไปถึง application ปลายทางก็จะมีกระบวนการ **Data Deserialization** เพื่อแปลงข้อมูลภาษากลางหรือ **binary data** นั้นให้กลับมาเป็น **object data** ได้ดั้งเดิมและถูกนำไปใช้ยัง application ปลายทางได้



การรับส่งข้อมูลระหว่าง APPLICATIONS บน MOBILE DEVICES





การรับส่งข้อมูลระหว่าง APPLICATIONS บน MOBILE DEVICES

- หรือตัวอย่างของการรับส่งข้อมูลบนโทรศัพท์มือถือที่เป็น **smart phone** เนื่องจาก Smart-phones แต่ละค่าย ก็จะมี ความแตกต่างกันทั้งในแง่ของการออกแบบระบบปฏิบัติการ และ application programming เช่น ระบบปฏิบัติการ **ios** จากค่าย **Apple** ที่ถูกใช้อยู่ใน **iPhone** และ **Android** จากค่าย **google** ที่ถูกใช้อยู่ใน **Samsung, Motorola, LG, HTC, Sony Ericsson** เป็นต้น
- สำหรับ **Applications** ต่างๆที่ถูกสร้างและใช้งานอยู่บนมือถือเช่น **email client, SMS program, calendar, maps, browser, contacts** และอื่นๆ จะใช้ภาษา **Java** ในการพัฒนา applications ต่างๆบนมือถือประเภทนี้ได้
- ซึ่งการแลกเปลี่ยนข้อมูลระหว่าง **applications** ในเครือข่ายทำให้ต้องมีการสร้างข้อมูลที่จะส่งให้อยู่ในรูปแบบโครงสร้างที่เหมาะสมโดยผ่านกระบวนการที่เรียกว่า **Data Serialization** ที่จะทำให้ข้อมูลถูกส่งไปยังปลายทางมีขนาดเล็ก และสามารถส่งได้อย่างรวดเร็ว



DATA SERIALIZATION FORMATS

- ❖ ในปัจจุบันจะแบ่งกลุ่มภาษาที่ถูกใช้เป็นมาตรฐานกลางของกระบวนการ Data Serialization เป็น 2 กลุ่มใหญ่ได้แก่
 1. รูปแบบที่เป็นข้อความที่มนุษย์สามารถอ่านได้ (text-based formats)
 - เช่น ภาษา XML, JSON, YAML ←
 2. รูปแบบของ binary data ที่มนุษย์ไม่สามารถอ่านได้ (binary-based formats)
 - เช่น BSON (Binary JSON), MessagePack, ProtoBuf (Protocol Buffers), Apache Thrift เป็นต้น



DATA SERIALIZATION FORMATS

- การจะเลือกภาษาสำหรับกระบวนการ data serialization สำหรับแอปพลิเคชันหนึ่งๆ จะขึ้นกับปัจจัยต่างๆ เช่น ความซับซ้อนของข้อมูล ความต้องการให้ผู้ใช้สามารถอ่านข้อมูลได้ด้วยหรือไม่ ความเร็วในการส่งข้อมูลและขนาดเนื้อที่จัดเก็บข้อมูลของคอมพิวเตอร์ หรืออุปกรณ์ทางคอมพิวเตอร์
- แต่ถ้าเปรียบเทียบ **text-based formats** กับ **binary-based formats** จะพบว่า **binary formats** จะมีขนาดที่เล็กมากกว่า **text-based formats** มากและมีความเร็วในการรับส่งข้อมูลมากกว่า **text-based formats**
- แต่ถ้าต้องการแลกเปลี่ยนข้อมูลผ่านทาง **Web services** ที่มีอยู่แล้วจะแนะนำให้ใช้ **text-based format** เพราะสามารถถูกเขียนและแปลบนหลากหลาย platform



TEXT-BASED DATA SERIALIZATION FORMATS

- สำหรับเนื้อหาในวิชานี้จะแนะนำให้รู้จักภาษาที่ถูกใช้ในกระบวนการ Data serialization ซึ่งเป็นภาษาที่มีลักษณะเป็นข้อความที่มนุษย์สามารถอ่านได้ ได้แก่ ภาษาต่อไปนี้
- XML (Extensible Markup Language)
 - JSON (JavaScript Object Notation)
 - YAML (Yet Another Markup Language)



XML (EXTENSIBLE MARKUP LANGUAGE)

- เป็นภาษาในตระกูล Mark up ที่เกิดจากภาษาต้นตระกูลที่ชื่อว่า **SGML (Standard Generalized Markup Language)** ถูกพัฒนาโดยองค์กร **W3C**
- ถูกออกแบบเพื่อให้ง่ายต่อการแลกเปลี่ยนข้อมูลระหว่าง applications บนเครือข่าย Internet และเป็นที่ยอมรับภาษาโปรแกรมหลากหลายภาษา
- มีโครงสร้างที่มนุษย์สามารถอ่านเข้าใจได้ และเป็นภาษาที่มีผู้คนรู้จักและใช้งานอย่างแพร่หลาย รวมทั้งมีการใช้กับ **Web services** ต่างๆ รวมทั้งเป็นพื้นฐานที่ถูกใช้สร้างเป็น **XML Applications** ต่างๆมากมาย



XML (EXTENSIBLE MARKUP LANGUAGE)

- อย่างไรก็ตาม **XML** จะมีการใช้คำอธิบายข้อมูลที่เรียกว่า **Elements** ต่างๆ ฟุ่มเฟือยมาก ทำให้ขนาดของเอกสาร XML มีขนาดใหญ่ซึ่งจะไม่เหมาะที่จะนำมาใช้รับส่งข้อมูลใน **mobile devices** ที่ต้องการข้อมูลที่มีขนาดเล็ก เพื่อให้สามารถส่งข้อมูลได้รวดเร็ว



XML EXAMPLE

```
<breakfast_menu>
  <food>
    <name>Belgian Waffles</name>
    <price>$5.95</price>
    <description>our famous Belgian Waffles </description>
    <calories>650</calories>
  </food>
  <food>
    <name>Strawberry Belgian Waffles</name>
    <price>$7.95</price>
    <description>Belgian waffles with strawberry</description>
    <calories>900</calories>
  </food>
</breakfast_menu>
```



JSON (JAVASCRIPT OBJECT NOTATION)

- เป็นรูปแบบการเก็บข้อมูลที่ถูกพัฒนาขึ้นโดย Douglas Crockford
- มีรูปแบบโครงสร้างการเก็บข้อมูลที่เล็กกว่า XML ซึ่งในปัจจุบันก็มีการใช้ JSON ใน Web services แล้ว
- JSON จะถูกมองว่าเป็น lightweight structure และมีประสิทธิภาพในการรับส่งข้อมูลที่ดีกว่า XML เนื่องจากจะไม่มีส่วนของ Elements ที่ฟุ่มเฟือยมากเท่า XML และได้ถูกพิจารณาเป็นภาษาที่สามารถถูกใช้กับ mobile devices ได้อย่างเหมาะสม
- อย่างไรก็ตาม JSON เองก็ยังมีข้อบกพร่องในบางเรื่อง เช่น ไม่รองรับเรื่อง namespace และ input validation เป็นต้น



JSON EXAMPLE

```
{ "breakfast_menu": [  
  { "name": "Belgian Waffles",  
    "price": "$5.95",  
    "description": "our famous Belgian Waffles",  
    "calories": "650"  
  },  
  { "name": "Strawberry Belgian Waffles",  
    "price": "$7.95",  
    "description": "Belgian waffles with strawberry",  
    "calories": "900"  
  }  
]}
```



YAML (YET ANOTHER MARKUP LANGUAGE)

- ได้ถูกเปลี่ยนชื่อภายหลังเป็น **YAML Ain't Markup Language** เนื่องจากต้องการจะสื่อว่าเป็นภาษาที่แตกต่างจาก Markup languages ทั่วไป เพราะจะมีโครงสร้างที่ง่ายกว่า
- ภาษานี้กำลังเป็นนิยมมากในปัจจุบัน เพราะจะมีรูปแบบคล้ายกับ XML และ JSON แต่ใช้ syntax ที่น้อยหรือง่ายกว่าและเป็นภาษาที่สั้นกว่าภาษา Markup อื่นๆ และรองรับ **Complex data structure** ด้วย
- จะถูกใช้บ่อยในการเขียน **server configuration** หรือ Apps ที่ต้องการเชื่อมข้อมูลกับฐานข้อมูล MySQL

YAML EXAMPLE

```
---
first_name: Adam
last_name: Bertram
hair_color: Brown
married: true
spouse:
  name: Miranda
  occupation: Mom
  interests:
    - Instagram
    - Facebook
    - "keeping the Bertram family in check"
dog_count: 2
dogs:
  dog1:
    name: Elliott
    breed: Shih-Tzu
    color: black/white
  dog2:
    name: Brody
    breed: Shih-Tzu
    color: black/white
```

← บอกจุดเริ่มต้นของการเขียนเอกสาร YAML

← โครงสร้าง Mapping ที่ประกอบด้วย **key:value** pairs

Value ของ key บางตัวก็สามารถอยู่ในรูปแบบ Mapping ได้อีก

Value ของ key บางตัวก็อาจอยู่ในโครงสร้าง list ที่เก็บข้อมูลคล้าย Array of values

Value ของ key บางตัวก็อาจมีโครงสร้างที่เรียกว่า dictionary ที่ประกอบด้วย list (dog1, dog2) ซึ่งแต่ละ list ก็ยังสามารถเก็บข้อมูลในลักษณะ Mapping (key:value pairs) ได้อีกด้วย



SUPPORTING PROGRAMMING LANGUAGES

- JSON, XML, YAML รวมไปถึง binary format เช่น `protobuf`, `MessagePack` จะได้รับการสนับสนุนจากหลากหลายภาษาโปรแกรมต่าง ๆ (programming languages) เช่น ภาษา `C`, `C++`, `Java`, `Python`, `Perl`, `Go`, และอื่นๆ
- ดังนั้นการจะเลือกภาษาใดก็จะขึ้นอยู่กับรูปแบบความต้องการและความเหมาะสมของงานด้วย ไม่ใช่เฉพาะแต่เรื่อง `serialization` อย่างเดียว



MLESTONES

ปีคศ.	ภาษาที่เกิดขึ้น
1972	CSV (Comma-Separated Values) และถูกนำมาใช้ในยุคของ modern PCs ตั้งแต่ปี 1983
1977	คำว่า Serialization ได้เกิดขึ้นเป็นครั้งแรกในภาษาจาวา โดยมีการสร้าง Serialization interface เพื่อแปลง Java object ที่เป็นข้อมูลผลลัพธ์ให้อยู่ในรูปแบบของ byte stream ที่จะทำให้สามารถเก็บลงใน Database ได้หรือส่งข้อมูลนั้นไปในเครือข่าย หรือเก็บในรูปแบบของ file เป็นต้น
1998	ได้เกิดภาษา XML 1.0 โดยการแนะนำของ W3C โดยถูกสร้างขึ้นมาจากภาษาดั้งตระกูล markup อันได้แก่ SGML
2001	Douglas Crockford ได้สร้าง JSON ขึ้นมาและเป็นที่นิยมในปี 2001
2004	YAML ซึ่งเป็น extension ของ JSON ได้ถูกสร้างขึ้นเป็นเวอร์ชันแรก



MLESTONES

ปีคศ.	ภาษาที่เกิดขึ้น
2008	-MessagePack ซึ่งเป็น binary serialization format ที่ได้มาจากการเกิดขึ้นของ JSON ซึ่งจะได้รับการ support จาก all major programming languages - Protocol Buffers จากค่าย Google ได้ถูกปล่อยออกมาเป็นทางการ
2016	BSON ถูกปล่อยออกมาในลักษณะของ independent data serialization format ภาษานี้เกิดขึ้นมาจริงๆในปี 2009 เพื่อใช้กับ MongoDB สำหรับใช้เก็บข้อมูลภายใน



NEXT CHAPTER

- Introducing XML
- Comparison of HTML vs XML
- Examples of XML Applications
- Examples of XML Editors
- XML Structure



REVIEW

1. อธิบายความหมายของ Data Serialization?



REVIEW

2. อธิบายความหมายของ Data Deserialization?



REVIEW

3. การแลกเปลี่ยนข้อมูลของ WWW ใช้ภาษาใดในการเก็บข้อมูลมาตรฐานกลางในการรับส่งข้อมูลระหว่างเครื่อง Client และ Server



REVIEW

4. ข้อเสียของ HTML คืออะไร



REVIEW

5. คำว่า Static Web page หมายถึงอะไร



REVIEW

6. การสร้าง Dynamic Web page คืออะไร มีข้อดีอย่างไร



REVIEW

7. การสร้าง Dynamic Web page จะมีภาษาอะไรบ้างที่สามารถถูกนำมาเก็บข้อมูลมาตรฐานกลางที่มนุษย์สามารถอ่านได้ เพื่อใช้ในการรับส่งข้อมูลระหว่างเครื่อง Client และ Server



REVIEW

8. การสร้าง Dynamic Web page จะมีภาษาอะไรบ้างที่สามารถถูกนำมาเก็บข้อมูลมาตรฐานกลางที่มนุษย์ไม่สามารถอ่านได้ เพื่อใช้ในการรับส่งข้อมูลระหว่างเครื่อง Client และ Server



REVIEW

9. ควรใช้ภาษาประเภทใดเป็นข้อมูลมาตรฐานกลางในการรับส่งข้อมูลระหว่าง Applications บน Mobile devices เพราะเหตุผลใด



REVIEW

10. ให้เรียงลำดับภาษาที่มีโครงสร้างที่ง่ายสุดไปยังซับซ้อนที่สุดระหว่าง JSON, XML, และ YAML